

Homework 5: NBA Player Social Graph (Tripti Singh)

Part 1 – summary

Part 2 – Final Social Graph

Data Used

For this project, I used the **NBA Per-36 Minutes dataset**, which contains 32 performance-related columns per player-season, including player name, season, team, position, and complete per-36 metrics (points, assists, rebounds, steals, blocks, shooting splits, and efficiency stats). I cleaned the dataset by removing duplicates, handling missing values, replacing NaNs with zeros, and verifying data types. After cleaning, the dataset contained **31,951 rows × 32 columns**, providing a reliable foundation for building the network.

From this full dataset, I selected **25 players** whose careers overlap across multiple teams and seasons. This group includes high-profile and multi-team players such as LeBron James, Kyrie Irving, Brandon Ingram, Julius Randle, John Wall, Bradley Beal, Stephen Curry, Kawhi Leonard, Blake Griffin, Kevin Durant, Damian Lillard, and others. This diverse subset provided a good mix of positions, playstyles, and teammate histories, making it well-suited for constructing a meaningful social graph based on weighted teammate connections.

How the Network Graph Was Constructed

Node Construction

I used Python to extract and prepare the node information for the network. Each of the 25 selected players was represented as a **single node**. For every player, I included detailed attributes such as name, position, team history, per-36 scoring averages, rebounding efficiency, assist rates, and other per-36 statistics. These attributes were compiled into a structured **node table** containing one row per player. I exported this table as **node_players.csv**, which served as the foundation of the network's node layer in Gephi.

In total, the node dataset consisted of **25 unique nodes**, each carrying a set of informative player-specific properties.

Edge Construction

To capture the relationships among the selected players, I constructed edges based on **shared NBA seasons and teams**. Using Python, I grouped the dataset by season and team to identify every roster composition for every year. For each team-season group, I generated all possible pairs of players who appeared together, treating each pair as a teammate connection.

If two players were teammates across multiple seasons, I accumulated these occurrences to compute an **edge weight**, which represents the strength of their relationship.

Examples from the final edge table include:

- **LeBron James - Kyrie Irving - weight = 6**
- **Lou Williams - Jrue Holiday - weight = 6**
- **Blake Griffin - Danilo Gallinari - weight = 8**

I exported all constructed relationships, including Source, Target, and Weight, into **edge_players.csv**, which holds the complete teammate network structure.

Graph Result

After constructing both tables, the final network consisted of:

- **Nodes:** 25
- **Edges:** 49
- **Edge Meaning:** Number of seasons two players played together

These weighted edges accurately reflect real NBA teammate histories and provide a meaningful basis for graph analysis and visualization.

3. Importing Node and Edge Files Into Gephi

After completing the Python-based processing, I switched to Gephi for visualization. I created a **new Gephi project** and began by importing **node_players.csv** as a **Nodes Table**. Gephi automatically recognized columns such as name, position, and all per-36 statistics, which I validated during the import process.

Next, I imported **edge_players.csv** as an **Edges Table**. During this step, I set the graph type to **Undirected**, because teammate relationships are mutual rather than directional. I also confirmed that the **Weight** column was properly assigned, ensuring that Gephi would treat edge thickness as a measure of relationship strength.

4. Final Data Validation in Gephi

Once both tables were imported, I opened the **Data Laboratory** in Gephi to verify that the dataset loaded correctly. The Data Laboratory displayed:

- **25 nodes**, each representing a single NBA player
- **49 weighted edges**, matching the number of teammate relationships generated in Python

With all nodes and edges accounted for and correctly attributed, the network was fully prepared for layout algorithms, centrality analysis, and final visualization.

How I Constructed and Visualized the NBA Player Social Graph

I selected NBA players and used Python to identify all teammate relationships based on shared team + season. Each player became a node, and every pair of players who appeared on the same roster formed an edge. If they played together in multiple seasons, those were added as an edge weight. I exported the node and edge tables to CSV and imported them into Gephi as an undirected, weighted network.

Below are the features, metrics, and layouts I experimented with to analyze and visualize the network :

Metrics Used for Size / Color

To style the network consistently across all seven visualizations, I used a fixed set of node and edge appearance rules:

Node Color (Ranking - for all the layouts)

- I used the *pink - white - green* gradient for node color.
- This helped highlight which players are embedded in strong, influential network regions.
- The same color scale was used across all centrality metrics to keep visuals comparable.

Edge Color (Ranking - Weight)

- For edges, I used the *red - yellow - blue* gradient.
- Red indicates low-weight teammate connections.
- Blue represents strong multi-year relationships.

I also used another edge color for Fruchterman Reingold Layout:

Blue - Teal - Light Yellow color gradient to encode edge weights.

- **Dark blue edges** represent weaker teammate relationships (fewer shared seasons).
- **Teal edges** represent moderate-strength relationships.
- **Light yellow edges** highlight players who shared **multiple seasons or multiple team overlaps**, making their connection visually stand out.

Node Size (Ranking - Different Metrics)

I varied **node size** depending on the metric being visualized:

Metric	Size Range	Used In
Eigenvector Centrality	22 - 55	ForceAtlas2, FR layouts
PageRank	22 - 55	ForceAtlas2
Modularity Class	22 - 55	ForceAtlas2
Closeness Centrality	1 - 4	Circular Layout
Betweenness Centrality	1 - 4	Fruchterman-Reingold
Clustering Coefficient	1 - 4	Fruchterman-Reingold
Weighted Degree	1 - 4	Fruchterman-Reingold

So:

- **Big nodes** = more influence / more connections (depending on the metric).
- **Small nodes** = lower structural importance.

Labels

I used one consistent label style:

Player Name – Position – G (games/team count)

Example: *Buddy Hield – SG – 5*

This makes comparisons across layouts easier.

Filtering

I tested filtering using:

- **Shortest Path**

- **Neighbors Network**
- **In/Out Degree Range**

This helped isolate sub-groups and check connectivity patterns (shown in the screenshot).

Algorithms / Layouts I Tried

Across Gephi, I generated seven final visuals using three main layout families.

1. ForceAtlas2 (Final layout for the main network)

- Best for social graphs.
- Captures weighted edges well.
- Pulls strong teammate groups together (e.g., Durant–Irving–Aldridge).
- Ideal for eigenvector, pagerank, and modularity visualizations.

2. Fruchterman–Reingold

Used for:

- Betweenness Centrality
- Clustering Coefficient
- Weighted Degree

Because :

- FR spreads nodes evenly.
- Makes size differences more visible.
- Easier to compare structural metrics without force-based clustering.

3. Circular Layout

Used only once for:

- **Closeness Centrality**

Because :

- Perfect circle = perfect symmetry
- Node size becomes the main focus
- Great for comparing closeness across all players
- No clustering bias – nodes are equally spaced

Short Explanation of My Design Choices

Colors

- **Node color:** gradient - shows influence

- **Edge color:** weight gradient - shows strength of teammate relationships
- Using the *same color scale* across all views keeps the analysis consistent.

Sizes

- Size directly tied to centrality metrics
- Larger = more influential / more central
- Smaller = peripheral or weaker connectors
- Size ranges were adjusted for readability (FR uses 1–4, FA2 uses 22–55).

Layouts

- **ForceAtlas2** chosen as the primary layout because it:
 - Separates communities clearly
 - Groups strong teammates naturally
 - Places hubs like Durant, Kyrie, Griffin in the center
- **FR layout** chosen for metric clarity
- **Circular layout** chosen for comparing closeness scores.

Filtering

- Shortest Path filter used to highlight relationships between two selected players.
- Neighbor-based filters helped confirm modularity structure and validate the layout.

Algorithms and Layouts Tried

To position the players meaningfully within the network, I tested several Gephi layout algorithms and selected **ForceAtlas2** as the final layout because it produced the clearest and most interpretable structure.

ForceAtlas2 (Layout Used)

ForceAtlas2 is a physics-based layout that simulates attraction and repulsion forces:

- Connected players **attract** each other
- All nodes **repel** each other to avoid overlap
- Stronger edges (higher weights) pull nodes **closer together**
- Communities naturally **separate into clusters**

I ran ForceAtlas2 with the following configuration:

- **Threads:** 13
- **Tolerance (Speed):** 1.0
- **Approximation:** 1.2
- **Approximate Repulsion:** Off

These settings produced a stable, well-spaced layout that clearly revealed structural patterns in the network. Core hubs such as **Kevin Durant**, **Kyrie Irving**, and **Blake Griffin** appeared centrally due to their many connections. Tight sub-communities, like the **Blake Griffin–Danilo Gallinari** group and the **LeBron James–Kyrie Irving** cluster, formed naturally through strong weighted edges. Meanwhile, players with fewer teammate links, such as **Lauri Markkanen** or **Jrue Holiday**, appeared more peripheral. Overall, ForceAtlas2 yielded the most intuitive and visually meaningful representation of the network.

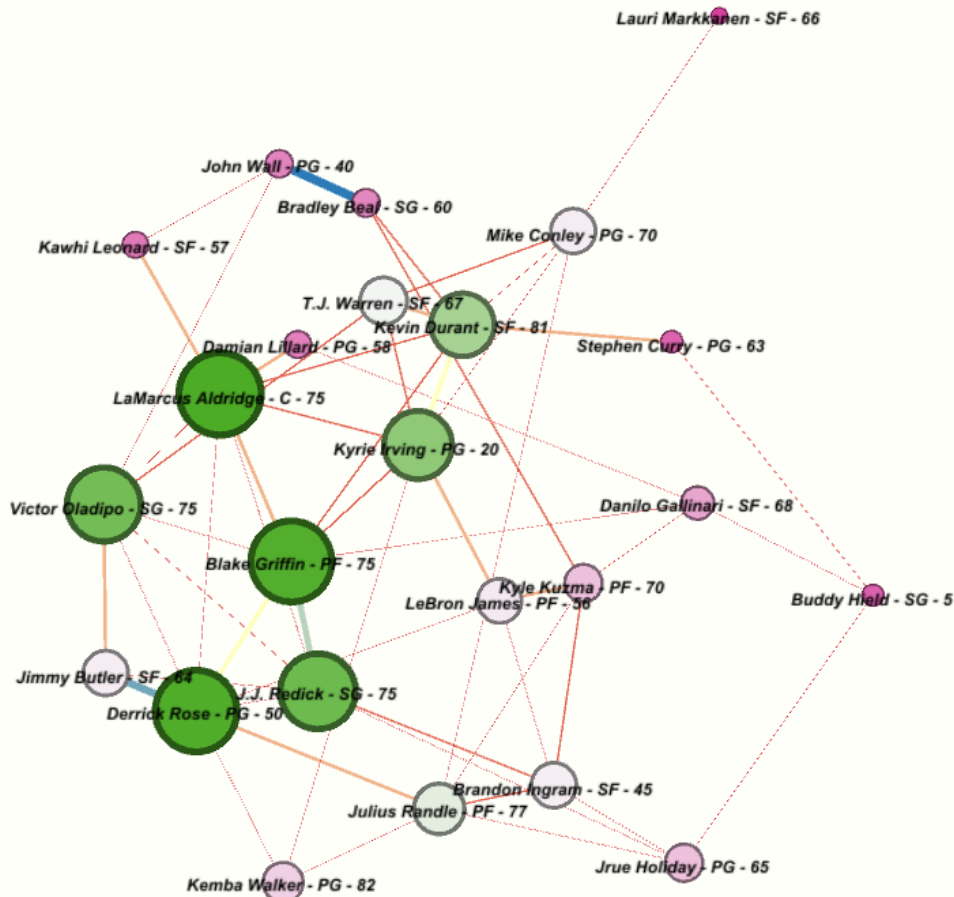
Metrics Used for Node Size, Node Color, and Edge Styling

Once the layout was established, I applied several centrality and community metrics in Gephi to highlight structure within the graph.

Eigenvector Centrality (Node Size)

I used Eigenvector Centrality to scale node sizes.

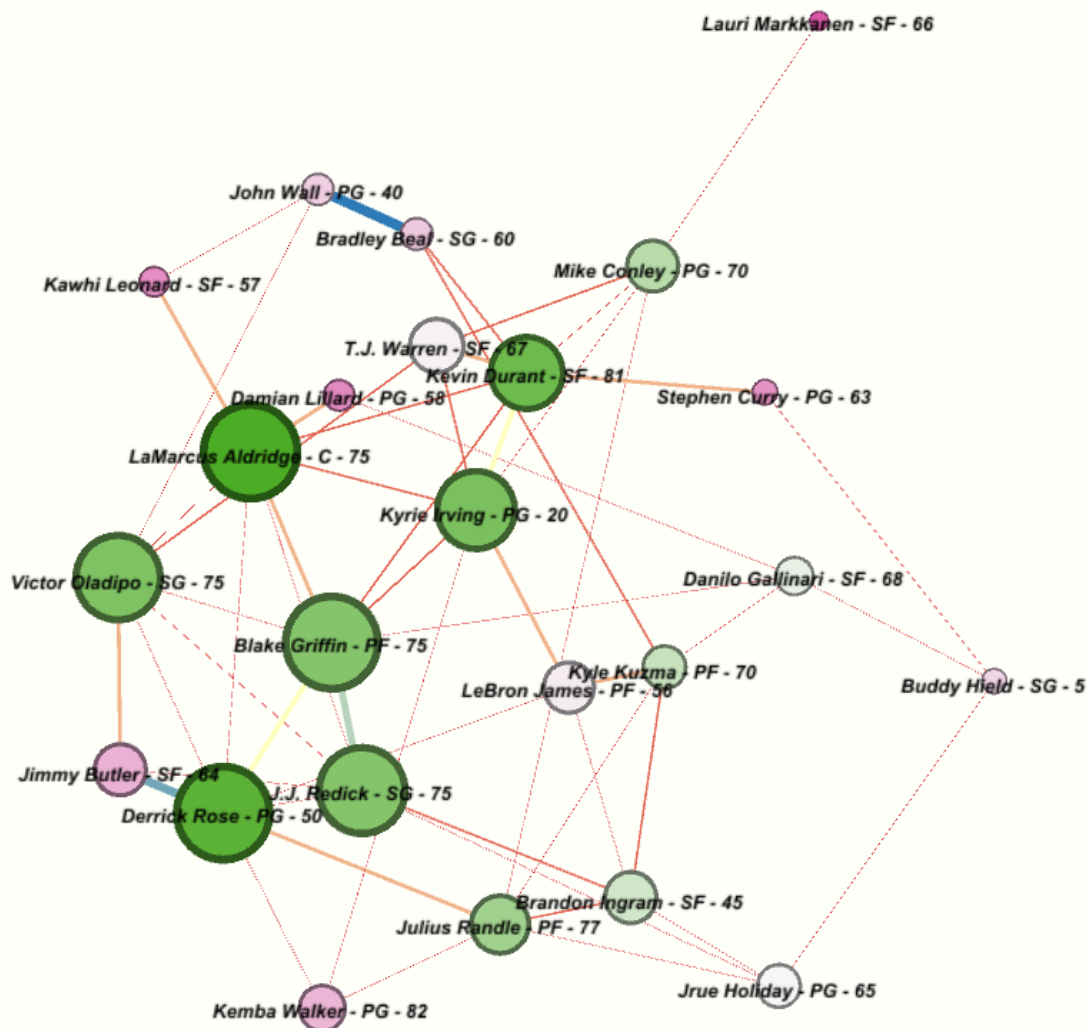
- **Larger nodes represent players who are not only well-connected, but connected to other highly influential players.**
- In this visualization, **Blake Griffin**, **LaMarcus Aldridge**, **Derrick Rose**, **Victor Oladipo**, and **Kevin Durant** appear larger because they sit in the strongest connectivity core of the network and have influential neighbors.
- Players positioned on the periphery (like **Buddy Hield** or **Lauri Markkanen**) remain smaller because their connections are fewer or linked to less influential nodes.



PageRank (Alternative Sizing Test)

I also experimented with PageRank as a secondary node-sizing metric.

- **PageRank increases for players who are connected to other players that are themselves influential**, even if the direct number of connections is smaller.
- In this visualization, PageRank assigns the highest values to players positioned in the network's core, which makes **Blake Griffin, LaMarcus Aldridge, Derrick Rose, Victor Oladipo, and Kevin Durant** appear larger.
- These players benefit from being part of dense sub-groups with many mutually influential teammates.
- Peripheral players such as **Buddy Hield, Lauri Markkanen, and Kemba Walker** receive lower PageRank scores and thus appear smaller.

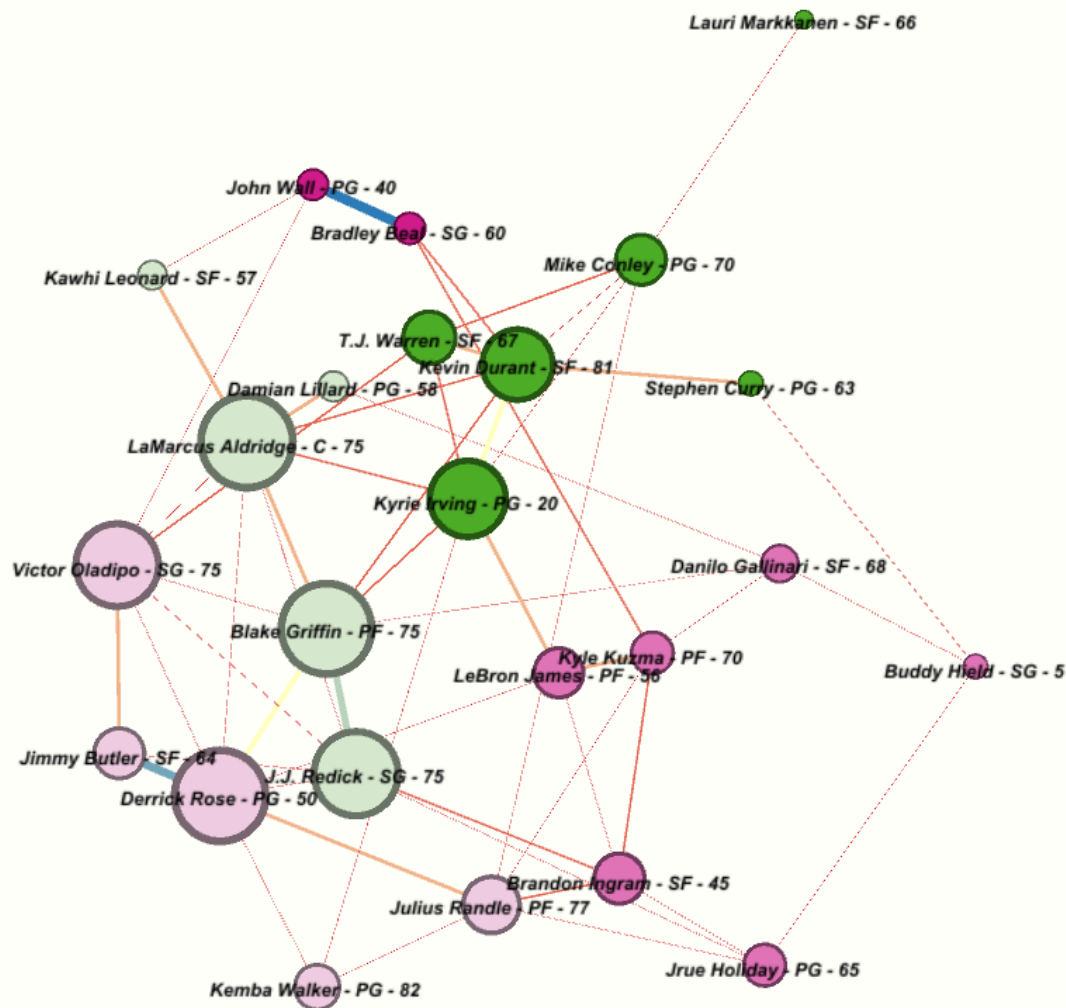


Modularity Class (Node Color)

To highlight natural communities in the network, I applied **Modularity Class** to color the nodes.

- Each community receives a **distinct color**, grouping players who share dense teammate relationships.
- In this visualization, modularity separates the graph into several meaningful clusters.
- One prominent cluster includes **Kevin Durant, Kyrie Irving, LaMarcus Aldridge, Blake Griffin, and J.J. Redick**, reflecting their overlapping team histories and strong internal connectivity.
- Another community groups players such as **Brandon Ingram, Julius Randle, Jrue Holiday, and Buddy Hield**, representing more loosely connected or perimeter players in the network.
- A lighter-colored community forms around **Victor Oladipo, Jimmy Butler, and Derrick Rose**, indicating a secondary cluster tied together by moderate-weight connections.

This color coding makes it easy to see how teammate patterns naturally form tightly-knit subgroups within the NBA player network.



Edge Weights in ForceAtlas2

In ForceAtlas2, **edge weights strongly control node attraction**. Since weight = number of seasons played together, players with long shared histories are pulled tightly toward each other.

- Thick, dark edges (e.g., Griffin–Gallinari, Durant–Irving) create **strong clusters**.
- Weak edges exert little pull, so loosely connected players stay farther apart.

Result: ForceAtlas2 forms **clear teammate communities** shaped directly by edge strength.

Additional Layout Tried: Fruchterman–Reingold

Along with ForceAtlas2, I also experimented with the **Fruchterman–Reingold** layout to explore a different spatial arrangement of the NBA player network. This layout is also force-directed, but it distributes nodes more evenly across the canvas, creating a circular or geometric structure rather than the tightly clustered look of ForceAtlas2.

Why I Tried Fruchterman–Reingold

Fruchterman–Reingold is useful for:

- Producing **balanced spacing** between nodes
- Reducing overlap when graphs are dense
- Highlighting **network symmetry**
- Visual clarity when showing metrics like Clustering Coefficient, Betweenness, or Weighted Degree

Because my graph contains many inter-connections (49 edges across 25 players), this layout helped visualize structural metrics more clearly.

Fruchterman–Reingold Settings Used

Based on my Gephi configuration (as shown in screenshots), I ran the layout using:

- **Area:** 10,000
- **Gravity:** 10
- **Speed:** 1.0

These settings forced nodes to spread out uniformly while still keeping connected players reasonably close.

Visual Insights From Fruchterman–Reingold

The layout revealed several patterns:

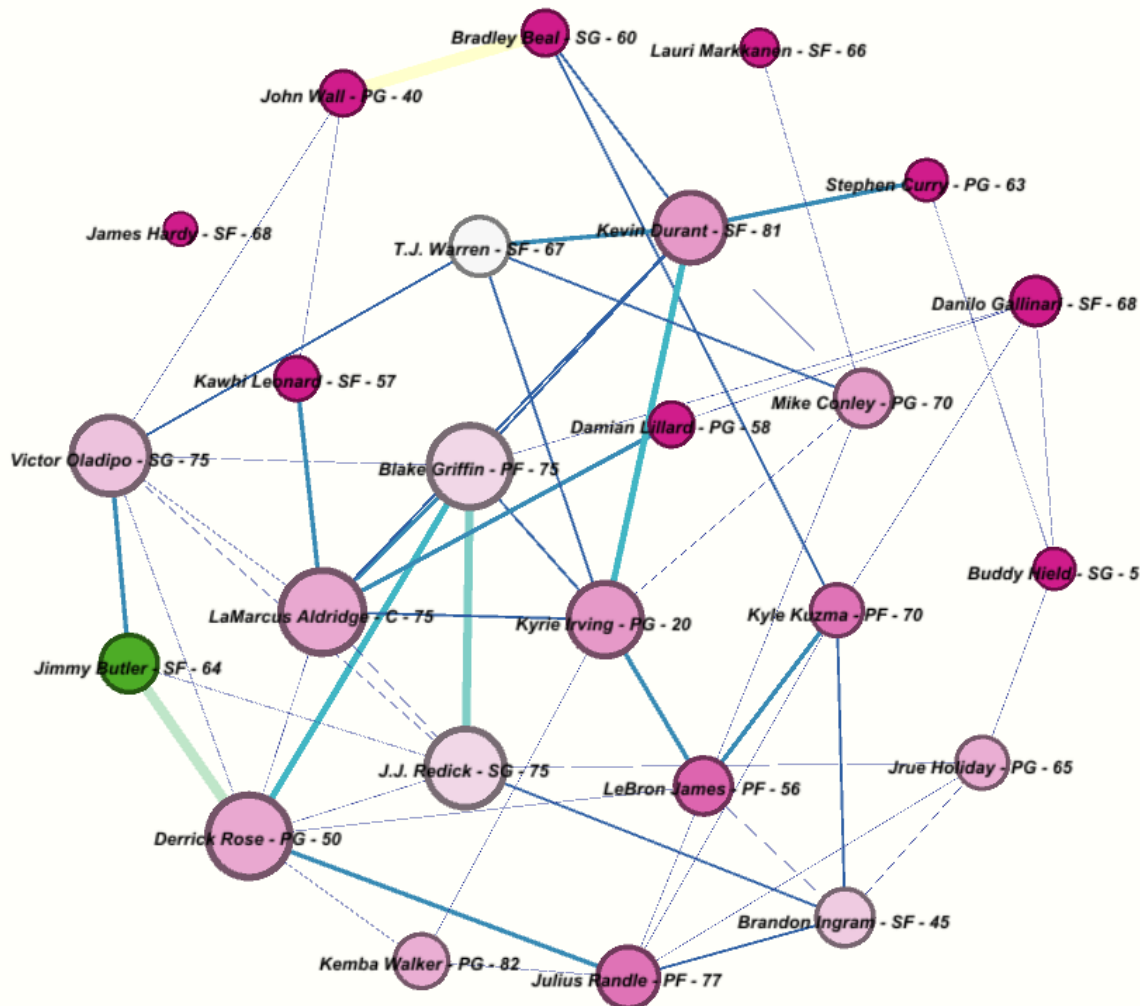
Clustering Coefficient (Node Size)

When I sized the nodes using **Clustering Coefficient**, players who are part of tightly interconnected subgroups became noticeably larger in the visualization.

- A high clustering coefficient means a player's teammates are also connected to each other (a dense triangle-like structure).
- This causes players in highly cohesive neighborhoods, such as **Blake Griffin, LaMarcus Aldridge, Kyrie Irving, and J.J. Redick**, to appear larger and more centered.

- These players sit inside the densest portion of the graph, where many strong teammate connections overlap, so their clustering values elevate naturally.

This makes tightly interconnected groups visually pop out, especially the **Griffin–Aldridge–Irving–Redick** cluster that sits at the structural core of the network.

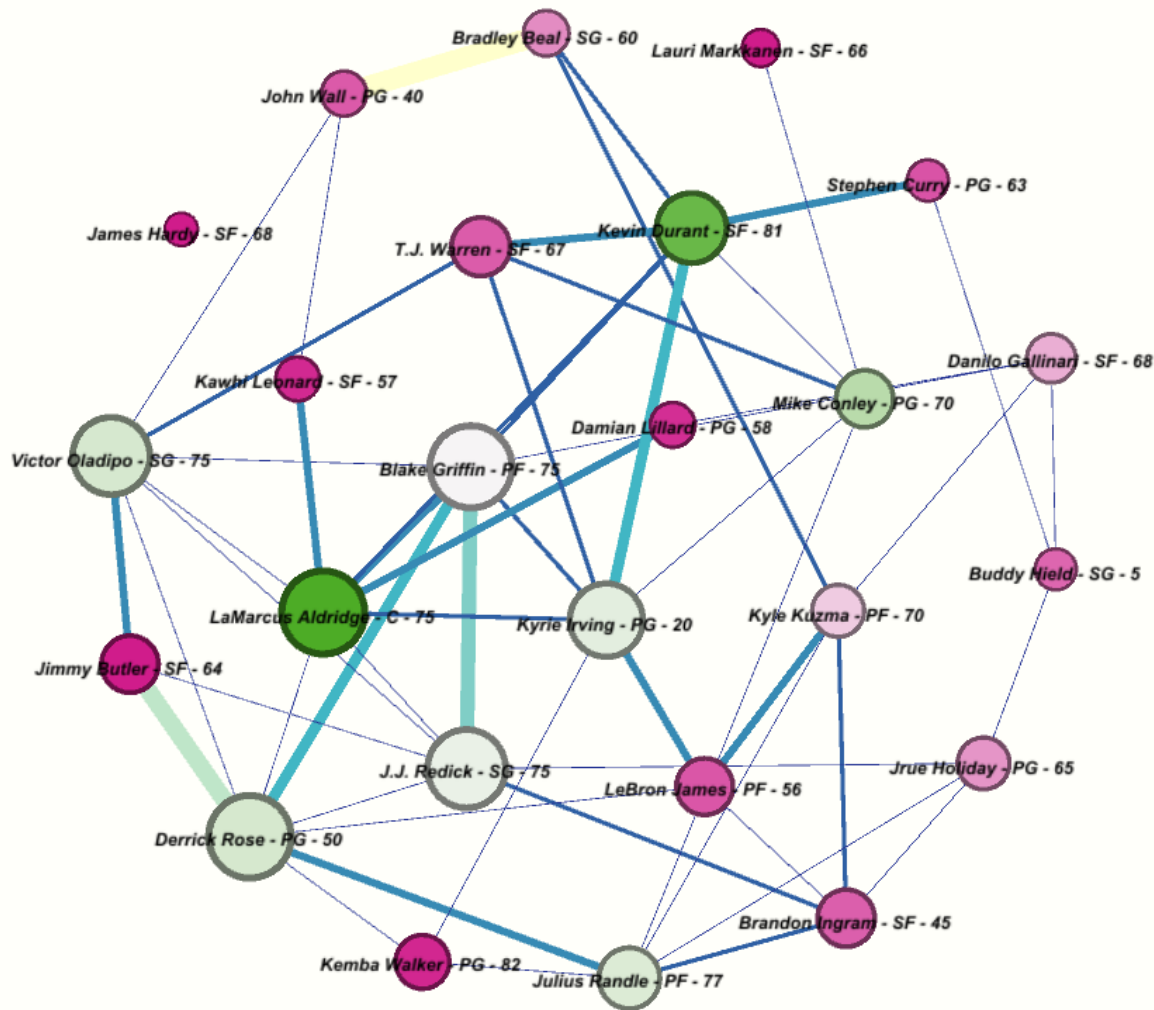


Betweenness Centrality (Node Size)

When I scaled the nodes using **Betweenness Centrality**, the Fruchterman–Reingold layout made the “bridge players” stand out clearly.

- Betweenness identifies players who frequently lie on the shortest paths between other players or sub-groups.
- In this visualization, **Kyrie Irving** becomes noticeably larger because he connects several high-weight clusters (Durant, Aldridge, Griffin, Kuzma, etc.).
- **Kevin Durant** also appears large, serving as a key intermediary linking upper and lower portions of the network.
- The FR layout spreads nodes evenly, preventing these central players from overlapping and making their bridging roles visually obvious.

This makes it easy to identify which players structurally “hold the network together.



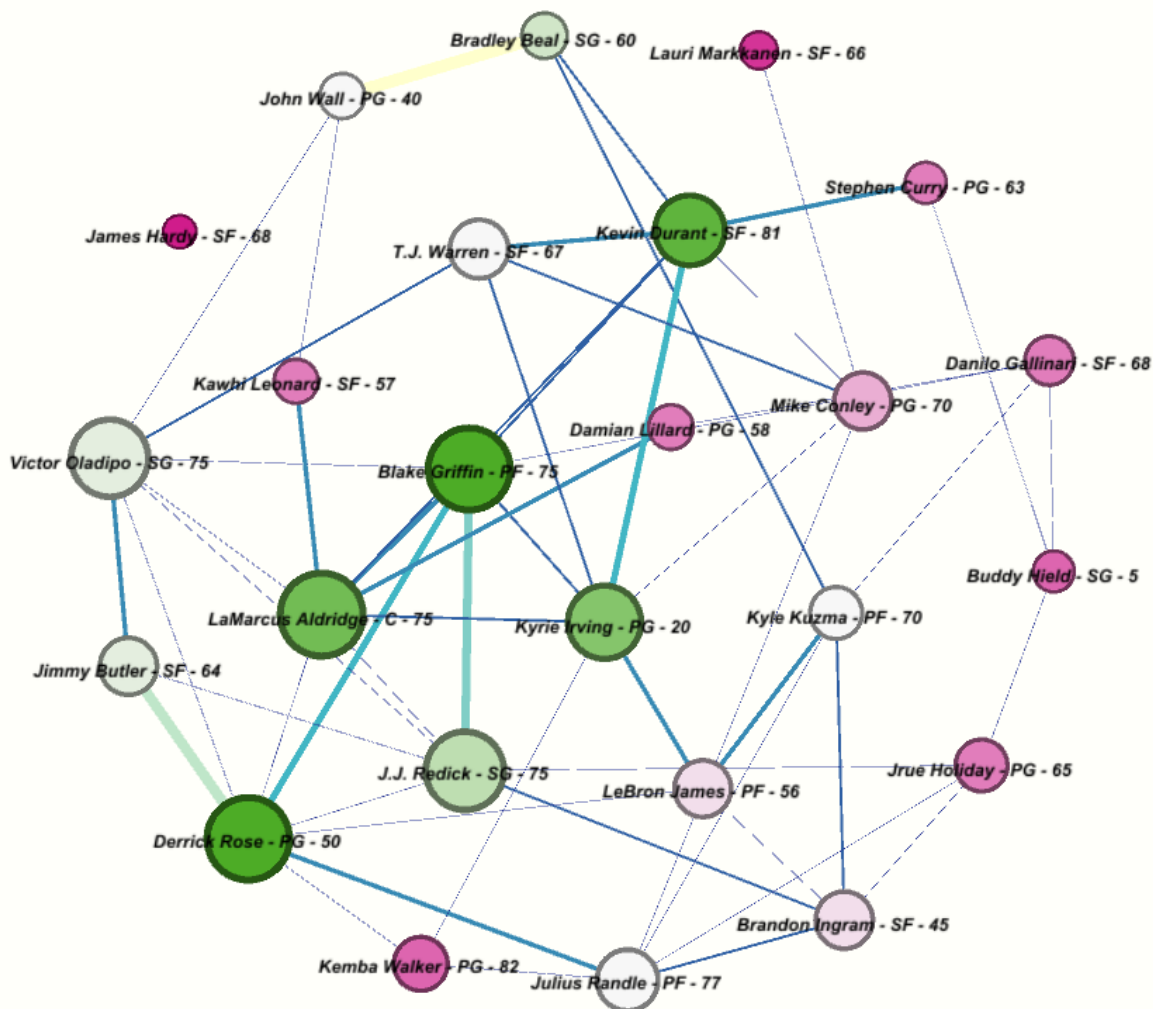
Weighted Degree (Node Size)

When I scaled the nodes using **Weighted Degree**, the Fruchterman–Reingold layout produced a very geometric and symmetrical structure. Weighted Degree highlights players who accumulated the *most total teammate connections*, considering both the number of links and the strength (edge weight) of each link.

In this visualization:

- **Blake Griffin, Kevin Durant, and Kyrie Irving** stand out as noticeably larger nodes because they share many high-weight connections across multiple seasons.
- Their large node sizes reflect how frequently they appeared with teammates and how central they are to roster overlap patterns.
- Because the FR layout distributes nodes evenly across the canvas, the **size differences become very easy to read**, without clutter or overlap.
- The uniform spacing also makes relationship intensity clearer, stronger connections radiate toward these high-degree players, forming a clean geometric structure.

Overall, the FR layout + Weighted Degree sizing gives a clear picture of which players dominated the network through repeated teammate relationships.

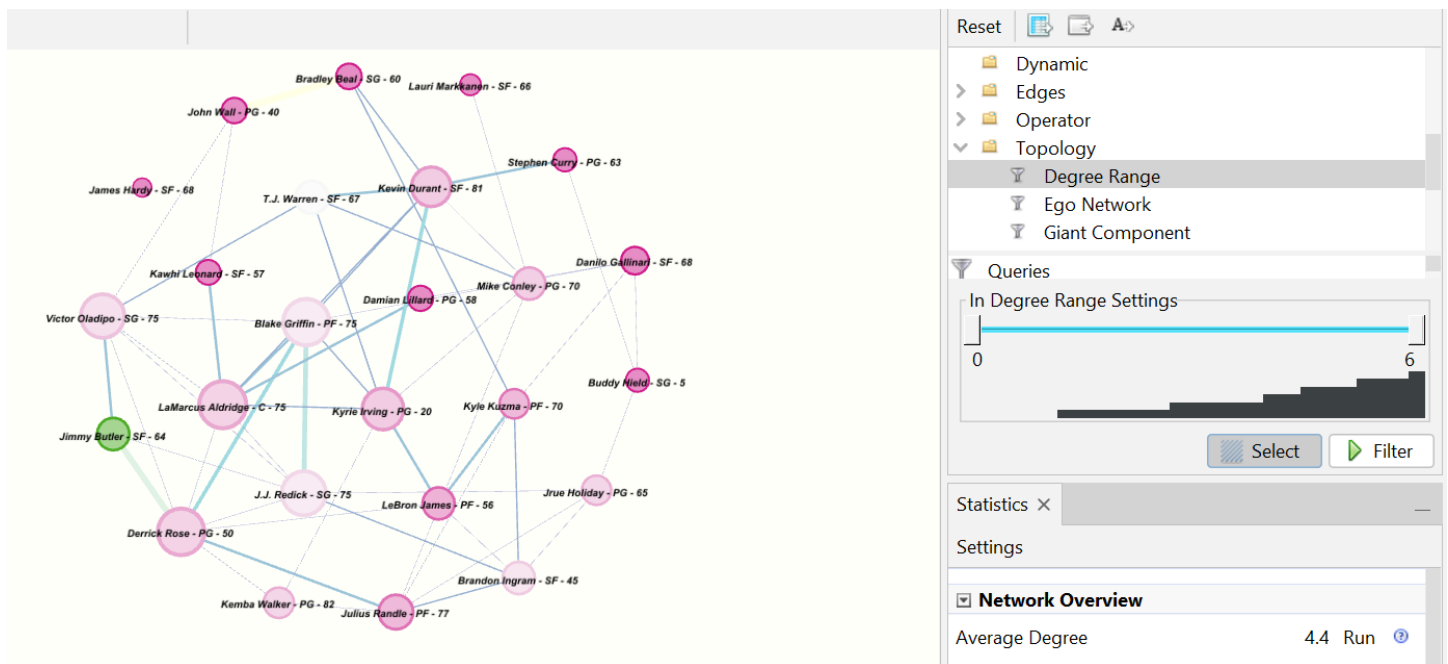


Edge Weights in Fruchterman–Reingold

In Fruchterman–Reingold, edge weights matter but **with weaker influence**.

- Thick edges still show strong relationships visually, but they don't collapse nodes together.
 - The layout stays **balanced and evenly spaced**, making weights easy to compare without distorting the graph.
- Result: FR highlights weights visually but keeps the structure **geometric and symmetrical**, not cluster-heavy.

Filtering: In Gephi, I used Degree Range, Neighbors, and Shortest Path filters to zoom into specific subgroups and verify connectivity between key players (e.g., Durant–Irving, Griffin–Gallinari)



Additional Layout Tried: Circular Layout

I also experimented with the **Circular Layout** in Gephi to view the network in a more structured, symmetric arrangement. Unlike force-directed layouts (ForceAtlas2 and Fruchterman–Reingold), the Circular Layout places all nodes evenly around a circle, making the graph highly organized and easy to scan. This layout is especially useful for comparing centrality metrics because node size differences become more visible when spacing is uniform.

Why I Tried the Circular Layout

The Circular Layout is ideal for:

- Presenting a **clean, non-overlapping structure**
- Showing **metric-driven node sizes** without distortion from forces
- Clearly comparing players' centrality values
- Visualizing dense connection patterns in a controlled shape

This layout essentially “fixes” the positions so the user can focus solely on metrics (size, thickness, communities) without the influence of physics forces.

Circular Layout Settings Used

Based on my Gephi configuration (shown in the screenshot), I used:

- **Diameter size:** 500
- **Order Nodes:** Node ID (ensuring stable ordering)
- **Node Layout Direction:** Counterclockwise

- **Prevent Node Overlap:** Enabled

These settings ensured even spacing and a clean circular shape.

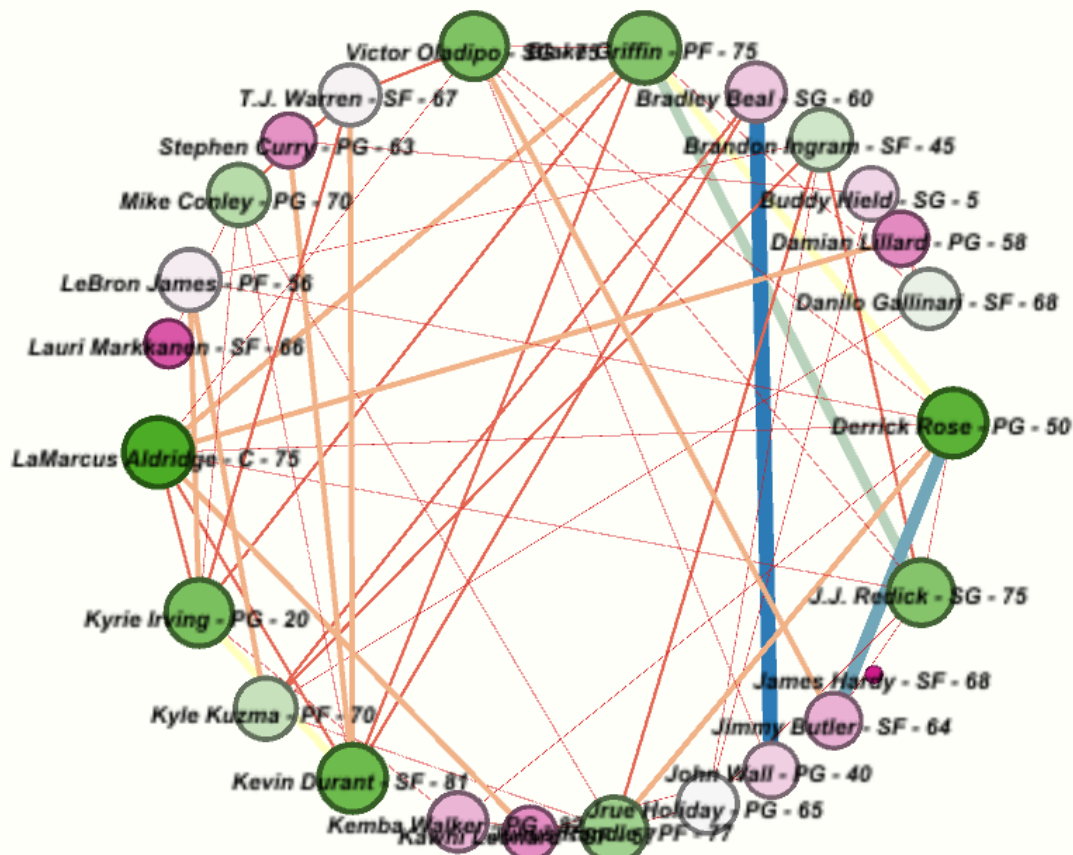
Visual Insights From the Circular Layout

Closeness Centrality Differences Become Very Clear

Since node positions are fixed, the size differences from **Closeness Centrality** stand out immediately:

- Larger nodes like **Kevin Durant**, **Kyrie Irving**, and **Blake Griffin** indicate strong closeness to the rest of the network.
- Smaller nodes such as **James Hardy**, **Buddy Hield**, and **Lauri Markkanen** appear on the circle with minimal visual distortion.

The uniform layout helps focus directly on centrality scores.



Edge Weights Form a Web-Like Pattern

Placed around a circle, the edges create an interconnected “spider-web” visual:

- Thick, dark edges (high weight) crisscross strongly in the center
- Thin, light edges fill the outer connecting paths

This makes it visually obvious which pairs shared many seasons (e.g., **Griffin–Gallinari**).

Excellent for Symmetry and Comparison

The circular layout makes it easy to compare:

- Player roles
- Node sizes across all centrality metrics
- Differences between high-degree and low-degree players
- How communities connect without spatial clustering bias

Unlike force-directed layouts, this view removes “interpretation noise” from gravitational or repulsive forces.

Which Layout Is Better - Final Comparison

After testing multiple Gephi layouts, **ForceAtlas2** emerged as the most effective and meaningful for this NBA player network.

1. ForceAtlas2 - Best Overall Layout

ForceAtlas2 is designed for social graphs, so it naturally pulls strongly connected players closer while pushing weaker connections apart. This makes:

- Teammate clusters very clear
- Central hubs (Durant, Irving, Griffin) easy to identify
- Edge weights strongly reflected in node positions

Best for: real interpretation, community structure, weighted networks like this one.

2. Fruchterman–Reingold : Good but Less Informative

Fruchterman–Reingold produces a clean, evenly spaced layout, but it reacts less to edge weights. Clusters appear, but they are softer and less distinct than in ForceAtlas2.

Strengths: visually neat, balanced spacing

Limitations: weaker cluster separation, less structural emphasis

3. Circular Layout : Great for Comparison, Not Interpretation

The Circular Layout organizes players in a perfect ring, making node size and color differences stand out immediately. However, it does not show the true structure of the network because distances do not reflect relationships.

Strengths: best for comparing centrality metrics

Limitations: no clustering, no structural meaning

Final Conclusion

ForceAtlas2 is the most representative and insightful layout for this social network. It uses edge weights effectively, highlights real teammate communities, and places key connector players in central positions. Fruchterman–Reingold is a solid secondary layout for visual clarity, while the Circular Layout is useful for metric comparison but not for understanding the actual network structure.

Additional Python Layout Tried: Spring Layout (Force-Directed)

I also experimented with NetworkX's **Spring Layout**, which is Python's default force-directed positioning algorithm. Similar to Gephi's force-based layouts, the Spring Layout simulates attraction (edges pulling nodes together) and repulsion (nodes pushing each other away). This layout attempts to naturally cluster players who share many connections.

However, unlike Gephi's physics engine, the Spring Layout is simpler and sometimes produces uneven or stretched positioning.

Why I Tried the Spring Layout

The Spring Layout is useful for:

- Quickly generating a basic force-directed graph
- Automatically clustering players based on shared edges
- Identifying hubs and dense subgroups
- Understanding the “shape” of the network without manual tuning

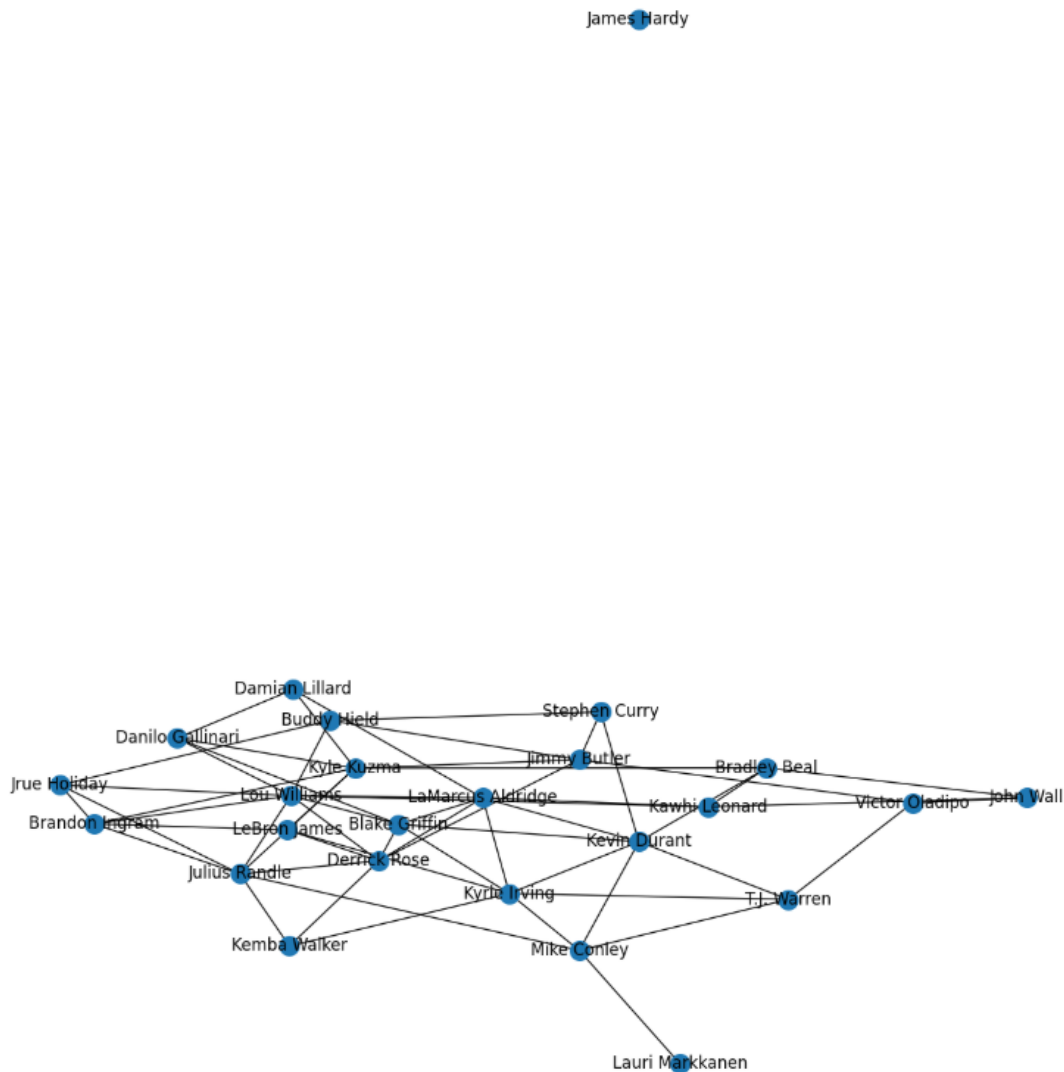
This layout is Python's closest equivalent to Gephi's ForceAtlas2, so testing it helps compare how much structure Python can capture.

Spring Layout Settings Used

Based on my Python configuration:

- seed = 42 for reproducible positioning
- Default repulsion/attraction parameters
- Node size: 200
- Font size: 12

These settings stabilize the layout while keeping labels readable.



Visual Insights From the Spring Layout

Kevin Durant and Kyrie Irving share one of the strongest connections in the network, having played multiple seasons together.

Their edge weight reflects repeated co-appearances on the same teams.

They sit near the center of the graph because they link several major sub-communities.

1. Natural Clusters Form, but Unevenly

Players like Kevin Durant, Blake Griffin, and Kyrie Irving cluster near the center due to multiple connections.

But some players, notably **James Hardy**, drift far from the network, creating large empty spaces.

2. Force Simulation Is Rough

The Spring Layout tends to stretch the graph vertically or horizontally:

- Dense regions collapse
- Sparse regions expand too far

- Labels sometimes overlap

3. Relationship Patterns Are Visible, but Messy

Although thick connectivity is visible, the layout lacks the refinement needed to clearly highlight:

- Strong teammate groups
- Modular communities
- Balanced graph structure

Overall, the Spring Layout gives a **quick but visually unstable** picture of the network.

Additional Python Layout Tried: Shell Layout (Circular-Ring Structure)

I also tested NetworkX's **Shell Layout**, which positions players in one or more concentric rings. Unlike force-directed layouts, the Shell Layout is geometric and symmetrical, creating an organized circular arrangement similar to Gephi's Circular Layout.

Why I Tried the Shell Layout

The Shell Layout is useful for:

- Creating a tidy, symmetric structure
- Ensuring all labels are visible
- Removing layout randomness and force distortion
- Comparing node attributes (degree, centrality) more easily

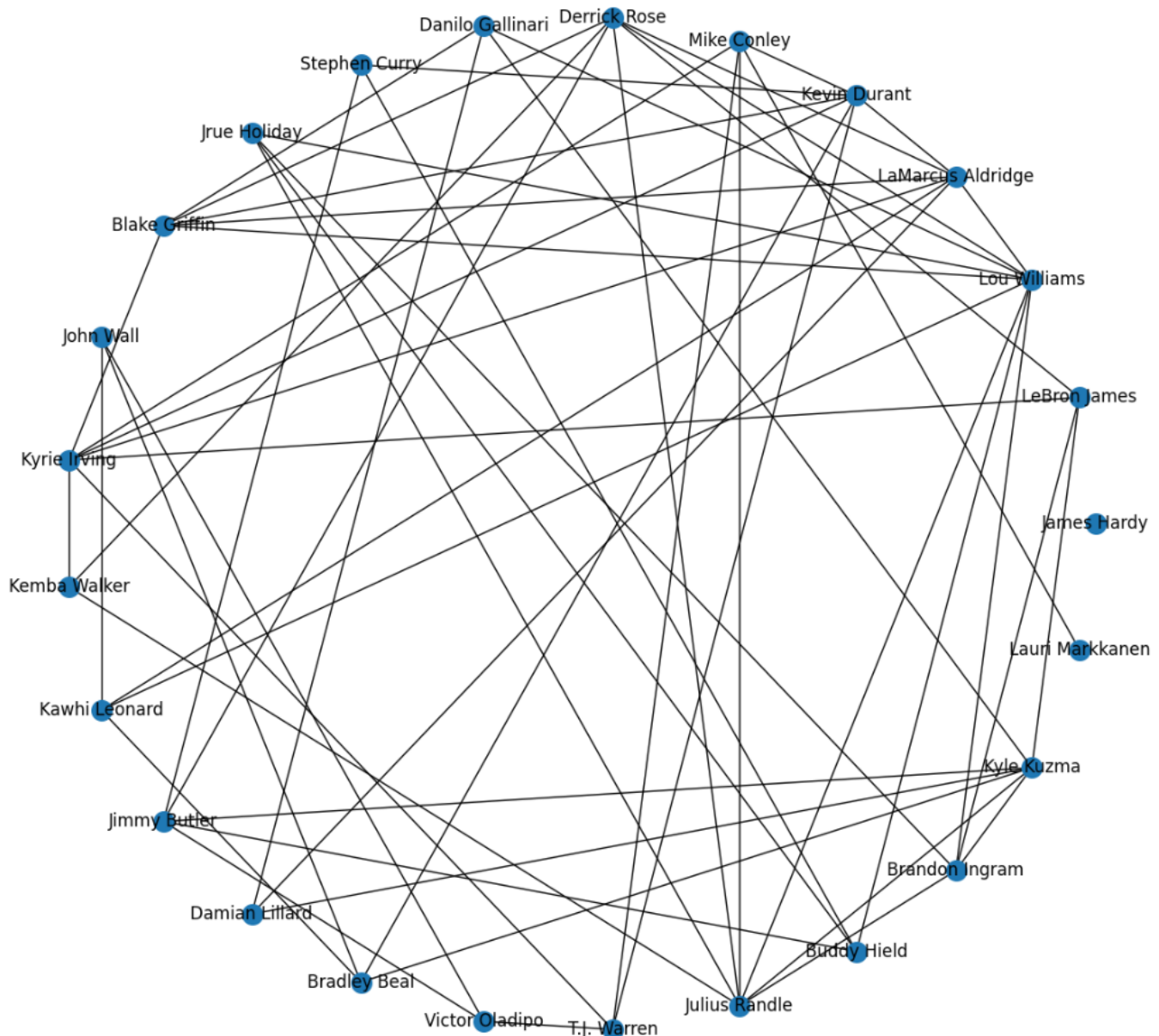
Because the positions are fixed in rings, the viewer can focus entirely on node sizes and connectivity.

Shell Layout Settings Used

From my Python configuration:

- Default single-ring placement
- Node size: 200
- Font size: 12

All nodes were positioned evenly around the circle.



Visual Insights From the Shell Layout

Blake Griffin and Danilo Gallinari form a tightly connected pair with a high edge weight. Their long shared history as teammates creates a strong pull between their nodes. This relationship helps anchor one of the major clusters in the network.

1. Perfect Symmetry but Low Structural Meaning

The layout looks very clean, but:

- Highly connected players may appear far apart
- Weakly connected players may sit adjacent

Because of this, network structure is **not accurately represented**.

2. Edge Connections Form a Dense Web

The circular shape causes many edges to crisscross:

- High-weight links overlap with low-weight links
- Dense regions become visually chaotic
- Hard to identify strong vs weak teammate relationships

3. Best for Size/Color-Based Comparisons

Despite the structural distortion, this layout is helpful for:

- Comparing centrality metric sizes
- Showing clear node labels
- Presenting a symmetric, non-force-based graph

The Shell Layout is aesthetically pleasing but **not ideal for interpreting NBA teammate relationships**.

Additional Python Layout Tried: Kamada–Kawai Layout (Energy Minimization)

I also applied the **Kamada–Kawai Layout**, which minimizes energy in the graph so that distances reflect edge relationships. This layout is more mathematically precise than Spring Layout and produces smoother, cleaner visuals.

Why I Tried the Kamada–Kawai Layout

This layout is excellent for:

- Producing balanced, symmetric graphs
- Preserving meaningful node distances
- Showing clusters without distortion
- Improving visual clarity compared to Python's Spring Layout

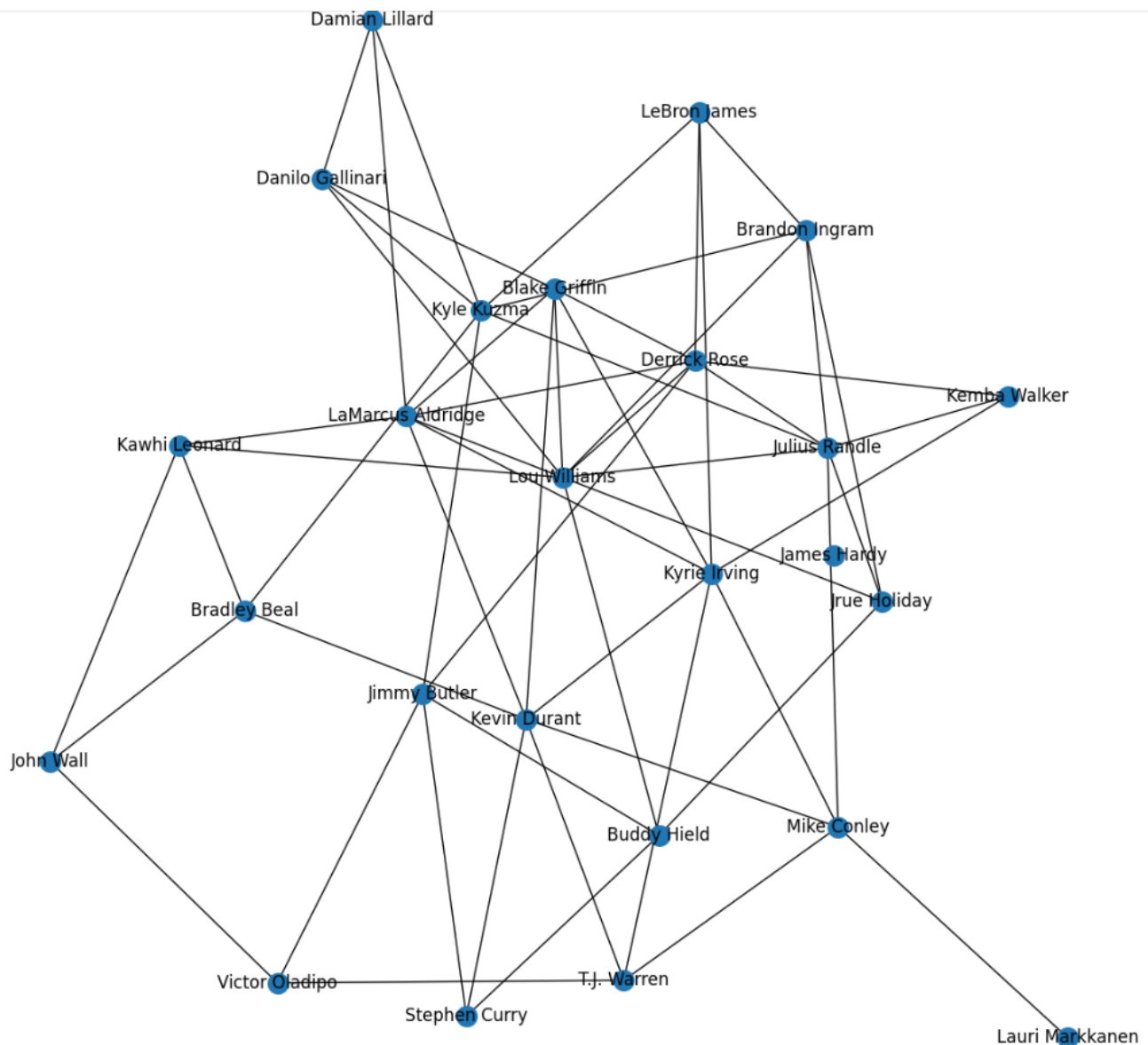
It is widely used when we need readable, structure-informed visuals from NetworkX.

Kamada–Kawai Settings Used

Using Python's default parameters:

- Node size: 200
- Font size: 12
- Full graph energy minimization

These settings produce a clean, well-distributed layout.



Visual Insights From Kamada–Kawai

LeBron James and Kyrie Irving have a strong multi-season teammate connection.

Their edge weight captures their overlapping years on powerful NBA rosters.

They appear close in most layouts, forming part of a highly interconnected core.

1. Best Overall Balance in Python

The layout avoids the large empty gaps seen in Spring Layout and the artificial circular shape of Shell Layout.

2. Clusters Form Naturally

Players with many teammate connections (e.g., Durant, Griffin, Irving) appear close together.

3. Labels and Edges Are Clearer

Edge crossing is reduced, making strong teammate clusters more visible.

This was the **most interpretable Python layout**.

Python Layout Summary - Which One Is Best?

Among the Python layouts, the Spring Layout offers natural force simulation but often becomes messy and uneven, making it visually unstable;

the Shell Layout provides perfect symmetry but lacks structural meaning, so it is useful only for simple comparisons.

the **Kamada–Kawai Layout delivers the most structured**, clean, and balanced visualization, though still not as advanced as Gephi. Overall, the Kamada–Kawai layout is the best Python option because it provides the strongest combination of clarity, structure, and readability.

Final Comparison: Gephi vs Python (Overall)

Why Gephi Is Better

Gephi visuals (ForceAtlas2, Fruchterman-Reingold, Circular, etc.) offer:

- Physics-accurate force layouts
- Thicker/darker edges based on real weights
- Modularity-based community colors
- Centrality-driven node sizing
- Label optimization
- Interactive tuning

Python cannot match this without extensive customization.

Final Verdict

ForceAtlas2 is the most representative and insightful layout for this social network. It uses edge weights effectively, highlights real teammate communities, and places key connector players in central positions.

Fruchterman–Reingold is a solid secondary layout for visual clarity, while the Circular Layout is useful for metric comparison but not for understanding the actual network structure.

Final Encodings Used in My Main ForceAtlas2 Graph

(after experimenting with multiple metrics such as PageRank, betweenness, closeness, clustering coefficient, and weighted degree)

- **Node Size = Eigenvector Centrality**
(final choice because it best highlights influential players across the full network)
- **Node Color = Modularity Class**
(clear visual grouping of teammate communities)

Edge Thickness + Edge Color = Edge Weight (seasons together)

(stronger teammate relationships appear thicker and darker)

Python layouts are helpful for testing, but Kamada–Kawai is the only one that approaches Gephi’s clarity , and still falls short for presentation-quality results.

Note: Please zoom in and zoom out in Gephi to clearly view the visuals.